Cachepawl Path C: Planner-Level Advisory Diagnosis of Hybrid Attention/Mamba Cache Overestimation

Formatting TODO: add venue-specific author and anonymization metadata

Venue-neutral draft

Abstract

Hybrid Attention/Mamba language models stress inference cache planners because attention KV pages and Mamba state caches have different shapes and reuse contracts. Cachepawl Path C evaluates whether these planner-level inefficiencies can be diagnosed from observed vLLM cache-plan artifacts without modifying the serving stack.

We present an observe/advisory workflow that captures vanilla `vllm==0.21.0` cache artifacts for `Zyphra/Zamba2-2.7B-instruct`, translates them into a Cachepawl schema, replays the vLLM planner stage on real planner inputs, and emits an advisory report through `cachepawl diagnose-vllm`. The planner-stage replay matched the runtime scheduler cache configuration and did not change the runtime scheduler state. Across a bounded 4-cell matrix for one model (`max_model_len` in {2048, 4096}, `gpu_memory_utilization` in {0.6, 0.7}, and `max_num_seqs=1`), estimated advisory savings ranged from 685,011,456 to 1,347,563,520 bytes. The overestimation ratio stayed 1.7333734577189286 and the wasted fraction stayed 0.4230902777777778 across completed cells.

We also include a separate RTX 3060 planner-only pack for synthetic hybrid KV/State workloads. In that deterministic pack, `cachepawl-avmp` stays closer to useful bytes than the `vllm-style-padded` planner baseline, but the result is not a vLLM serving measurement.

This is a diagnostic/advisory systems artifact. It does not claim runtime cache substitution, allocator replacement, serving-time VRAM reduction, throughput improvement, latency improvement, accuracy improvement, or quality impact.

1 Introduction

Hybrid Attention/Mamba language models place two different cache shapes behind one inference runtime. Attention layers use KV blocks whose economics are tied to token sequence growth, while Mamba or SSM layers use state tensors whose useful byte footprint can diverge from the page shape used by a uniform cache planner. Prior serving systems such as vLLM's PagedAttention [2] and SGLang [4] provide the systems context for this problem, while hybrid model families such as Jamba [3] and Zamba2 [1] motivate the mixed cache-shape setting. A cache planner that reserves by a shared page/block abstraction can therefore allocate capacity according to a shape that is larger than the useful state payload for hybrid layouts.

Cachepawl Path C studies this gap as an observe/advisory problem. The goal of this phase is not to replace vLLM's allocator, rewrite live runtime state, or report serving results. Instead, the goal is to answer a narrower systems question: can Cachepawl observe a vanilla vLLM hybrid cache plan, translate it into a stable schema, replay the planner stage on the observed inputs, and emit a practical diagnostic report that quantifies planner-level overestimation?

1.1 Scope of This Version

This version is scoped to evidence-backed diagnosis. It includes a bounded Path C matrix for one observed `Zyphra/Zamba2-2.7B-instruct` setup, a separate deterministic RTX 3060 planner-only comparison for synthetic `jamba-1.5-mini` workloads, and the `cachepawl diagnose-vllm` artifact-input CLI. It excludes runtime allocator replacement, runtime cache substitution, live VRAM reduction, serving throughput, serving latency, quality, accuracy, and controlled substitution readiness.

The answer from the current artifact set is yes, within a bounded scope. The evaluation uses `Zyphra/Zamba2-2.7B-instruct` under vanilla `vllm==0.21.0` and a 4-cell advisory matrix with `max_model_len` in {2048, 4096}, `gpu_memory_utilization` in {0.6, 0.7}, and `max_num_seqs=1`. Across all four completed cells, the planner-level overestimation ratio stayed 1.7333734577189286 and the planner-level wasted fraction stayed 0.4230902777777778. Estimated advisory savings ranged from 685,011,456 to 1,347,563,520 bytes.

Those values are advisory metrics over planner artifacts. They are not runtime VRAM measurements, throughput measurements, latency measurements, accuracy measurements, or quality measurements. The phase deliberately keeps artifact-input diagnosis separate from controlled substitution.

1.2 Contributions

The contributions of this report are evidence-bounded:

- a bounded method for capturing and translating vLLM hybrid cache-plan artifacts;
- planner-stage replay evidence that matches the runtime scheduler config;
- a 4-cell Path C advisory matrix showing planner-level over-reservation for one `Zyphra/Zamba2-2.7B-instruct` setup;
- a deterministic RTX 3060 planner-only comparison showing `cachepawl-avmp` stays closer to useful bytes than a uniform padded planner baseline on three synthetic hybrid workloads;
- a user-facing `cachepawl diagnose-vllm` artifact-input report that is read-only and non-mutating;
- an explicit contract checklist that keeps controlled substitution in future work until Mamba state-index and state tensor paths are observable or supported upstream.

2 Method

The Path C method is an observe/translate/replay/diagnose workflow. It is designed to preserve the boundary between advisory analysis and runtime mutation.

2.1 Observe Vanilla vLLM Artifacts

The workflow starts from a vanilla `vllm==0.21.0` run for `Zyphra/Zamba2-2.7B-instruct`. Observation scripts collect cache-plan artifacts and safe runtime metadata. The observation path is read-only: it does not edit vLLM source, monkeypatch vLLM internals, replace allocators, or return Cachepawl plans to vLLM.

The captured artifacts include the runtime cache plan, safe metadata about the planner inputs, scheduler/KV manager structure, worker tensor layout, live request/block assignment, and attention-side block-table/view metadata.

2.2 Translate Cache Plans

The runtime cache plan is translated into a Cachepawl schema that separates the fields needed for advisory metrics from the fields that would be required for future controlled substitution. The translation records cache group count, cache tensor count, layer count, block count, per-group cache kind, page size, block size, useful bytes, and observed reserved bytes.

2.3 Replay the Planner Stage

The workflow replays `vllm.v1.core.kv_cache_utils.get_kv_cache_configs` on real planner inputs captured from the observed run. The replay is bounded and post-initialization. It is used to check whether the planner-stage translation matches the runtime scheduler cache configuration. The current evidence records:

- `planner_matches_runtime_scheduler=true`;
- `runtime_changed_during_replay=false`.

2.4 Compute Advisory Metrics

Cachepawl compares the observed reserved cache bytes with the useful byte footprint implied by the translated hybrid cache plan. The diagnostic report records observed reserved bytes, observed useful bytes, Cachepawl recommended bytes for the advisory planner view, estimated advisory savings bytes, overestimation ratio, and wasted fraction. These are planner-level advisory metrics. They estimate over-reservation in the observed planner artifacts. They are not serving-time VRAM measurements and do not imply throughput, latency, accuracy, or quality effects.

2.5 Package Artifact-Input Diagnosis

`cachepawl diagnose-vllm` is the productized output of the phase. It consumes an existing translated cache config and optional raw safe metadata, then writes `report.json`, `summary.md`, and `manifest.json`. The artifact-input CLI is intentionally lightweight. It does not require vLLM, GPU access, CUDA, NVML, or model loading. It does not modify vLLM, replace allocators, or enable runtime mutation.

2.6 Inspect Mutation Contracts

The final method step is not substitution. It is a contract audit that records which runtime structures were safely observable and which were not. The current evidence observed planner-stage replay match, request-to-block assignment, worker tensor layout, and attention block-table/view metadata. Mamba state-index and Mamba state tensor contracts remain blocked.

3 Evaluation

The evaluation has two evidence tracks. The Path C track measures planner-level over-reservation in observed `vllm==0.21.0` cache-plan artifacts for `Zyphra/Zamba2-2.7B-instruct`. The RTX

3060 planner-comparison track uses a deterministic synthetic planner harness for `jamba-1.5-mini` workloads. Both tracks are advisory/planner-level only.

3.1 Procedure and Non-Mutation Controls

The Path C workflow observes a vanilla vLLM cache plan, translates it into the Cachepawl schema, replays the vLLM planner on captured inputs, runs `cachepawl diagnose-vllm`, and repeats the advisory diff over the bounded 4-cell matrix. Planner-stage replay succeeded with `planner_matches_runtime_scheduler=true` and `runtime_changed_during_replay=false`.

All observations were read-only. The workflow did not modify vLLM, monkeypatch private methods, replace allocators, return Cachepawl plans to vLLM, alter scheduler behavior, or alter worker tensor layout. This validates the advisory comparison against the observed planner/runtime config, but it does not establish runtime substitution safety.

The planner-comparison pack is reproduced with:

```
UV_CACHE_DIR=/tmp/uv-cache uv run python \
  benchmarks/scripts/create_planner_comparison_pack.py \
  --output-dir benchmarks/results/rtx3060/planner-comparison \
  --seed 1 --num-requests 128 \
  --gpu-name "NVIDIA GeForce RTX 3060" \
  --gpu-total-bytes 12884901888
```

The pack does not install vLLM, run serving, monkeypatch vLLM, replace allocators, load a real model, run kernels, or measure latency and throughput.

3.2 Metric Definitions

- `reserved_bytes`: bytes reserved by the observed or modeled planner.
- `useful_bytes`: cache payload bytes before planner padding.
- `estimated_savings_bytes = reserved_bytes - useful_bytes` for Path C.
- `overestimation_ratio = reserved_bytes / useful_bytes`.
- `wasted_fraction = (reserved_bytes - useful_bytes) / reserved_bytes`.
- `virtual_oom`: whether a planner estimate exceeds the 12 GiB target profile in the synthetic planner pack. It is not an observed runtime OOM.

3.3 Path C Advisory Matrix

Table 1 is advisory/diagnostic evidence only. It does not report runtime mutation, throughput, serving, or VRAM improvement measurements.

Across all four completed cells, the overestimation ratio stayed 1.7333734577189286 and the wasted fraction stayed 0.4230902777777778. Estimated advisory savings ranged from 685,011,456 to 1,347,563,520 bytes. The full matrix, including `num_blocks`, cache group count, cache tensor count, layer count, and status, is recorded in `research/avmp/v2/evaluation/matrix_table.md`.

max_model_len	gpu_memory_utilization	Reserved	Useful	Savings
2048	0.6	1,893,335,040	1,092,283,392	801,051,648
2048	0.7	3,185,049,600	1,837,486,080	1,347,563,520
4096	0.6	1,619,066,880	934,055,424	685,011,456
4096	0.7	2,910,781,440	1,679,258,112	1,231,523,328

Table 1: Path C advisory matrix for `Zyphra/Zamba2-2.7B-instruct`. Values are bytes.

3.4 RTX 3060 Planner-Only Comparison

The planner-comparison pack reports deterministic planner estimates for three synthetic hybrid KV/State workloads. The `vllm-style-padded` backend models a uniform padded cache planner, not exact vLLM internals. The `cachepawl-avmp` backend models a native KV-page plus SSM-block planner. Table 2 is copied from `benchmarks/results/rtx3060/planner-comparison/summary.md`. This is planner evidence for the cache-shape claim, not runtime evidence for vLLM serving behavior.

Workload	Backend	Useful	Estimated	Over.	Waste	OOM
short-heavy	vllm-style-padded	2,435,055,616	6,985,613,312	2.868770	0.651418	false
short-heavy	cachepawl-avmp	2,435,055,616	2,451,046,400	1.006567	0.006524	false
long-heavy	vllm-style-padded	41,967,550,464	165,116,116,992	3.934376	0.745830	true
long-heavy	cachepawl-avmp	41,967,550,464	41,983,672,320	1.000384	0.000384	true
mixed	vllm-style-padded	13,517,422,592	51,311,017,984	3.795917	0.736559	true
mixed	cachepawl-avmp	13,517,422,592	13,532,397,568	1.001108	0.001107	true

Table 2: RTX 3060 planner-only comparison. Useful and estimated columns are bytes.

This supports a narrow planner claim: when the synthetic workload contains heterogeneous KV and state-cache shapes, AVMP's planner model keeps estimated bytes close to useful bytes, while the uniform padded planner model reserves substantially more bytes. The result does not prove live serving memory reduction or request-admission improvement. The pack does not run vLLM serving, and `long-heavy` plus `mixed` remain virtual-OOM even under AVMP because useful demand itself exceeds the 12 GiB target profile.

3.5 Runtime Contracts and Claim Boundary

Runtime contract observations resolved scheduler/KVCacheManager structure, block usage metadata, worker tensor layout, live request-to-block assignment, attention block-table/view metadata, and attention metadata builders. The Mamba side remains gated: `mamba_state_idx` was reachable but empty for the live request, no Mamba state tensors were safely reachable by stable runtime attributes, and runtime cache config reported `mamba_cache_mode: none`.

These blockers are decisive for this phase. Without Mamba state-index and state tensor contracts, the evidence supports observe/advisory reporting only. The evidence does not show runtime allocator replacement, runtime cache substitution, measured runtime VRAM reduction, throughput improvement, latency improvement, quality improvement, or accuracy improvement.

4 Limitations

This report is scoped to a diagnostic/advisory systems artifact. Its main limitations are:

- no runtime mutation: the evaluation does not replace vLLM allocators, rewrite cache views, alter scheduler behavior, alter worker tensor layout, or return Cachepawl plans to vLLM;
- no serving performance or quality measurement: the evidence does not include runtime VRAM reduction, throughput, latency, serving, quality, or accuracy experiments;
- narrow Path C scope: one observed model, vanilla `vllm==0.21.0`, `max_model_len` in {2048, 4096}, `gpu_memory_utilization` in {0.6, 0.7}, and `max_num_seqs=1`;
- narrow planner-comparison scope: synthetic `jamba-1.5-mini` workloads and a deterministic RTX 3060 target profile, not real model inference;
- local platform scope: the Path C artifacts are bounded to the recorded local RTX 3060 12 GiB / WSL2 context and should not be generalized to production serving clusters.

The Mamba-side mutation contracts remain blocked. The observed runtime cache config reported `mamba_cache_mode: none`, so this run did not populate the Mamba state-index/state-tensor paths needed for a view rewrite contract. Because these contracts are blocked, the report cannot claim controlled substitution readiness.

Future work must first improve Mamba state-index observability. A controlled substitution experiment should only be considered after Mamba state paths are observable through bounded read-only runtime paths, a supported vLLM integration seam exists, or a default-off mutation probe with rollback controls and parity checks is available. The next evaluation step is multi-model and multi-workload advisory evaluation, still without conflating advisory savings with measured runtime improvements.

5 Conclusion

Cachepawl Path C supports a narrow but useful systems claim: planner-level hybrid cache over-reservation can be diagnosed from committed vLLM cache-plan artifacts without modifying the serving stack. The current evidence includes a bounded `Zyphra/Zamba2-2.7B-instruct` Path C matrix, a deterministic RTX 3060 planner-only comparison, and an artifact-input diagnostic CLI. Together these support advisory reporting, not runtime replacement.

The technical ambition remains controlled substitution for hybrid cache planning, but that is future work. Before this paper can claim live VRAM reduction, throughput, latency, quality, accuracy, or allocator replacement, it needs Mamba state-index and state tensor contracts, a default-off substitution probe, and live serving measurements with rollback and parity controls.

A Artifact and Reproducibility Checklist

All paths are relative to the repository root. This checklist maps each report claim to committed evidence and records the commands needed to verify the venue-neutral LaTeX package.

A.1 Claim Artifacts

Artifact	Claim supported

`research/avmp/v2/results/vllm-runtime-cache-diagnostic-cli/report.json`	User-facing advisory report with planner-level reserved/useful/savings metrics.
`research/avmp/v2/results/vllm-runtime-cache-diagnostic-cli/summary.md`	Human-readable advisory summary and non-mutation statement.
`research/avmp/v2/results/vllm-runtime-cache-diagnostic-cli/manifest.json`	Artifact-input mode, output list, and non-requirement flags for vLLM, CUDA, GPU, and NVML.
`research/avmp/v2/results/vllm-planner-stage-observation/translated_planner_stage_config.json`	Planner-stage replay output translated into Cachepawl's schema.
`research/avmp/v2/results/vllm-planner-stage-advisory-diff/diff_report.json`	Planner-stage advisory metrics and mutation-prevention field list.
`research/avmp/v2/results/vllm-planner-stage-advisory-diff/group_level_diff.json`	Per-cache-group advisory diff details.
`research/avmp/v2/evaluation/matrix_table.md` and `matrix_table.csv`	Four-cell advisory matrix and deterministic machine-readable metrics.
`benchmarks/results/rtx3060/planner-comparison/`	Deterministic RTX 3060 planner-only comparison pack.
`research/avmp/v2/evaluation/claim_summary.md`	Paper-facing claim boundary and unsupported claims.

A.2 Reproducibility Checklist

Item	Status	Evidence or command
Path C claim boundary documented	Ready	`research/avmp/v2/evaluation/claim_summary.md`
Path C matrix table committed	Ready	`research/avmp/v2/evaluation/matrix_table.md`, `research/avmp/v2/evaluation/matrix_table.csv`
Planner-comparison pack committed	Ready	`benchmarks/results/rtx3060/planner-comparison/`
Planner-comparison pack regeneration	Ready	Command listed below.
Diagnostic CLI tests	Ready	Command listed below.
Planner evidence tests	Ready	Command listed below.
Artifact path check	Ready	Confirm every path listed in this appendix exists in the repository.
Venue-specific formatting	TODO	Select target template, convert to the venue format, and add required author metadata and bibliography style.

Planner-comparison pack regeneration command:

```
UV_CACHE_DIR=/tmp/uv-cache uv run python \
  benchmarks/scripts/create_planner_comparison_pack.py \
  --output-dir benchmarks/results/rtx3060/planner-comparison \
  --seed 1 --num-requests 128 \
  --gpu-name "NVIDIA GeForce RTX 3060" \
  --gpu-total-bytes 12884901888
```

Diagnostic CLI test command:

```
UV_CACHE_DIR=/tmp/uv-cache uv run pytest \
  tests/cli/test_diagnose_vllm.py -q
```

Planner evidence test command:

```
UV_CACHE_DIR=/tmp/uv-cache uv run pytest \
  tests/bench/test_planner_comparison.py \
  tests/bench/test_vllm_path_c_advisory_matrix.py -q
```

A.3 Submission Readiness

Ready: venue-neutral LaTeX draft with title, abstract, introduction, method, evaluation, limitations, conclusion, artifact map, and reproducibility checklist; compact evaluation tables for the Path C matrix and RTX 3060 planner-only comparison; explicit advisory/planner-level claim boundary throughout the paper; committed evidence paths for numeric claims and product artifacts.

Remaining before venue submission: choose the target venue format and page limit; convert this simple article structure into the target template; add venue-required author, anonymization, ethics, artifact, and bibliography metadata; decide whether to keep the full artifact checklist in the main paper, appendix, or supplemental material; run the venue-specific build once the target template exists.

References

[1] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, Anna Golubeva, Vasudev Shyam, James Whittington, Jonathan Pilault, and Beren Millidge. The Zamba2 suite: Technical report. *arXiv preprint arXiv:2411.15242*, 2024.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 2023.

[3] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avshalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[4] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.